

Progetto Rientr@Returns
CNR-STIIMA

Documentazione Tecnica

`rdb2rdf_step2.py`

Matching tra DB relazionale e ontologia
tramite String Matching e modelli NLP

Versione	Step 2
Linguaggio	Python 3.10+
Ontologia	Rientra.rdf (namespace JobList)
Database	PostgreSQL 17

Indice

1	Introduzione e contesto	3
1.1	Scopo del modulo	3
1.2	Posizione nel progetto	3
2	Architettura e struttura del codice	4
2.1	Panoramica della pipeline	4
2.2	Organizzazione del file	4
2.3	Dipendenze	5
2.3.1	Dipendenze obbligatorie	5
2.3.2	Dipendenze opzionali per NLP completo	5
3	Configurazione	6
3.1	Parametri globali	6
3.2	Soglie di confidenza	6
4	Sorgenti dati	8
4.1	Database relazionale esterno	8
4.2	Ontologia Rientra.rdf	8
4.2.1	Namespace rilevanti	8
4.2.2	Professioni O*NET presenti	9
4.2.3	Estrazione dinamica	9
5	Strategie di matching	10
5.1	Strategia 1 – String Matching	10
5.1.1	Descrizione	10
5.1.2	Metriche	10
5.1.3	Implementazione	11
5.2	Cosine Similarity – funzione condivisa	11
5.3	Strategia 2 – BERT base	12
5.3.1	Modello	12
5.3.2	Strategia di embedding	12
5.3.3	Limitazioni: anisotropy	13
5.4	Strategia 3 – BioBERT	13
5.4.1	Modello	13
5.4.2	Ragione dell’inclusione	13
5.4.3	Risultati osservati	13
5.5	Strategia 4 – MiniLM	14
5.5.1	Modello	14

5.5.2	Differenze rispetto a BERT raw	14
5.5.3	Implementazione	14
5.5.4	Input testuale	15
6	Produzione dei risultati in formato Excel	16
6.1	Scopo dell’output Excel	16
6.2	Struttura del Workbook	16
6.3	Dettaglio del foglio “Matching Results”	16
6.3.1	Metadati e colonne	16
6.3.2	Logica del Verdetto	17
6.4	Stili e Formattazione Condizionale	17
6.5	Esempio di implementazione	18
7	Output e interpretazione dei risultati	19
7.1	Tabella comparativa	19
7.2	Risultati correnti	19
8	Limitazioni note e sviluppi futuri	21
8.1	Limitazioni correnti	21
8.1.1	Vocabolario ontologico insufficiente	21
8.1.2	Soglie BERT non calibrate	21
8.1.3	BioBERT – dominio errato	21
8.2	Sviluppi pianificati	21
9	Guida all’uso	22
9.1	Prerequisiti	22
9.2	Installazione	22
9.3	Esecuzione	22
9.4	Output atteso	22
A	Messaggi di warning noti	24
B	Glossario	25

Capitolo 1

Introduzione e contesto

1.1 Scopo del modulo

Il modulo `rdb2rdf_step2.py` costituisce il secondo passo del processo RDB2RDF del progetto **Rientr@Returns**, sviluppato presso CNR-STIIMA. L'obiettivo è risolvere il *mismatch* tra le denominazioni dei lavori presenti in un database relazionale esterno (`ext_job`) e le professioni rappresentate nell'ontologia principale Rientr@, costruita sul vocabolario O*NET.

Il problema è formalmente definibile come un task di *entity linking*: dato un'entità (job title + descrizione) proveniente da una sorgente eterogenea, trovare la classe ontologica corrispondente nell'ontologia di riferimento, tenendo conto che i nomi possono essere completamente diversi pur descrivendo la stessa professione.

Nota

Esempio concreto del mismatch: il job title “*Manager of the digital archive*” nel DB relazionale corrisponde alla classe O*NET “*Archivists*” nell'ontologia. I due nomi non condividono nessuna parola, ma la descrizione delle attività è semanticamente equivalente.

1.2 Posizione nel progetto

Il modulo si colloca tra due fasi:

- **A monte:** la tabella `ext_job` in PostgreSQL, creata con `rientra_ext_jobs.sql`, contenente 9 professioni con titolo e descrizione.
- **A valle:** un file excel (`rientra_matching.xlsx`), che mostra i lavori del db con i match trovati nell'ontologia; vengono mostrati anche i risultati prodotti dalle strategie implementate per trovare i match.

Capitolo 2

Architettura e struttura del codice

2.1 Panoramica della pipeline

Il modulo esegue le seguenti fasi in sequenza:

1. **Setup:** connessione a PostgreSQL, caricamento dell'ontologia RDF, estrazione dinamica delle professioni O*NET.
2. **Lettura DB:** recupero dei job dalla tabella `ext_job`.
3. **Strategia 1:** String Matching lessicale (Jaccard + Overlap + Levenshtein).
4. **Strategia 2:** NLP con BERT base, embedding [CLS], cosine similarity.
5. **Strategia 3:** NLP con BioBERT, embedding [CLS], cosine similarity.
6. **Strategia 4:** NLP con MiniLM tramite `sentence-transformers`, cosine similarity.
7. **Confronto:** tabella comparativa dei risultati con verdetto di accordo tra modelli.
8. **Output Excel:** creazione del file excel per la visualizzazione dei match trovati.

2.2 Organizzazione del file

Il file è organizzato in sezioni logiche separate da commenti delimitatori. La tabella seguente riassume le funzioni principali con il loro ruolo.

Funzione	Righe approx.	Ruolo
<code>score_color()</code>	87–93	Mappa score → stile rich
<code>score_bar()</code>	94–96	Genera barra testuale ASCII
<code>get_connection()</code>	103–104	Apri connessione PostgreSQL
<code>fetch_all()</code>	106–110	Esegue query, restituisce lista dict
<code>test_connection()</code>	112–122	Verifica connettività DB
<code>extract_ontology_jobs()</code>	129–160	Estrae professioni dall'ontologia
<code>_build_onto_text()</code>	162–169	Costruisce testo composito O*NET
<code>_norm()</code> , <code>_jaccard()</code> ,	176–201	Metriche string matching

Funzione	Righe approx.	Ruolo
<code>_overlap()</code> , <code>_levenshtein()</code> <code>_combined()</code>	200–201	Score pesato S1
<code>s1_match_one()</code>	203–215	Match singolo job con S1
<code>run_string_matching()</code>	217–227	Esegue S1 su tutti i job
<code>cosine_similarity_matrix()</code>	234–253	Cosine similarity vettorializzata
<code>get_bert_embedding()</code>	260–288	Embedding [CLS] con BERT raw
<code>run_bert_matching()</code>	291–349	Matching con BERT base o BioBERT
<code>run_minilm_matching()</code>	356–407	Matching con MiniLM
<code>print_ontology_table()</code>	414–425	Tabella rich professioni ontologia
<code>print_db_table()</code>	428–438	Tabella rich job nel DB
<code>print_full_comparison()</code>	456–559	Tabella comparativa strategie
<code>print_summary_multi()</code>	609–639	Pannello riepilogativo
<code>set_header()</code>	679–686	Genera i setting
<code>style_cell()</code>	689–694	Stile per il file excel
<code>save_excel()</code>	697–935	Salva il file excel
<code>main()</code>	722–847	Orchestrazione pipeline

Tabella 2.1: Funzioni principali del modulo

2.3 Dipendenze

2.3.1 Dipendenze obbligatorie

```
1 pip install psycpg2-binary rdflib scikit-learn rich numpy
```

Listing 2.1: Installazione dipendenze minime

2.3.2 Dipendenze opzionali per NLP completo

```
1 pip install sentence-transformers transformers torch
```

Listing 2.2: Installazione dipendenze NLP

Nota

Il modulo implementa *graceful degradation*: se `transformers` o `sentence-transformers` non sono installati, le relative strategie vengono saltate silenziosamente e l'output contiene solo le strategie disponibili.

Capitolo 3

Configurazione

3.1 Parametri globali

Tutti i parametri modificabili sono raccolti nella sezione di configurazione in cima al file, in modo da non richiedere modifiche al codice funzionale per adattare il modulo a un ambiente diverso.

```
1 DB_CONFIG = {
2     "host":      "localhost",
3     "port":      5432,
4     "dbname":    "rientra_db",
5     "user":      "postgres",
6     "password":  "postgres",
7 }
8
9 ONTOLOGY_FILE = "Rientra.rdf"
10 OUTPUT_DIR    = "output"
11
12 JOBLIST = Namespace("http://www.stiima.cnr.it/JobList#")
13 RIENTRA = Namespace("https://www.stiima.cnr.it/rientra#")
14
15 S1_THRESHOLD    = 0.12
16 BERT_THRESHOLD  = 0.70
17 MINILM_THRESHOLD = 0.50
18
19 MODELS = {
20     "S2_BERT":      "bert-base-uncased",
21     "S3_BioBERT":   "dmis-lab/biobert-base-cased-v1.2",
22     "S4_MiniLM":    "all-MiniLM-L6-v2",
23 }
```

Listing 3.1: Sezione di configurazione

3.2 Soglie di confidenza

Le soglie controllano quando un match viene considerato valido. Un match con score inferiore alla soglia viene salvato come `score_raw` nell'output visivo ma **non viene inserito nel grafo RDF** come tripla di collegamento.

Strategia	Parametro	Valore attuale	Note
S1 String Match	S1_THRESHOLD	0.12	Bassa: metriche già combinate
S2 BERT base	BERT_THRESHOLD	0.70	Da alzare a 0.92 (anisotropy)
S3 BioBERT	BERT_THRESHOLD	0.70	Da alzare a 0.92 (anisotropy)
S4 MiniLM	MINILM_THRESHOLD	0.50	Calibrata, affidabile

Tabella 3.1: Soglie di confidenza configurate

Attenzione

Le soglie di BERT base e BioBERT sono attualmente fissate a 0.70, ma i risultati sperimentali mostrano che con questi modelli tutti i 9 job superano la soglia indipendentemente dalla qualità del match, per via del fenomeno di *anisotropy* (si veda la Sezione [5.3.3](#)). Il valore raccomandato dal referente è ≥ 0.90 .

Capitolo 4

Sorgenti dati

4.1 Database relazionale esterno

La tabella `ext_job` in PostgreSQL costituisce la sorgente dei job da matchare. Ha struttura intenzionalmente minimale: tre colonne che rappresentano l'identificatore, il titolo e la descrizione del lavoro.

```
1 CREATE TABLE ext_job (  
2     id          VARCHAR(6) PRIMARY KEY, -- ext01, ext02, ...  
3     title       TEXT      NOT NULL,  
4     description TEXT  
5 );
```

Listing 4.1: Schema della tabella `ext_job`

I 9 job attualmente presenti spaziano su domini molto diversi: contabilità, gestione contenuti web, data science, relazioni pubbliche digitali, operazioni di segreteria e vendita al dettaglio. Questa eterogeneità è intenzionale e serve a testare i limiti del vocabolario ontologico corrente.

4.2 Ontologia Rientra.rdf

L'ontologia viene letta dal file `Rientra.rdf` in formato RDF/XML. Contiene 12.250 triple che definiscono la T-Box (classi, proprietà, relazioni) del modello Rientra@.

4.2.1 Namespace rilevanti

Prefisso	URI
JobList:	http://www.stiima.cnr.it/JobList#
rientra:	https://www.stiima.cnr.it/rientra#
01:	http://www.stiima.cnr.it/01:

Tabella 4.1: Namespace principali dell'ontologia

4.2.2 Professioni O*NET presenti

L'ontologia contiene attualmente 13 professioni nel namespace `JobList`, estratte dalla classificazione O*NET:

URI locale	Label O*NET
<code>Billing_cost_and_rate_clerks</code>	Billing, Cost, and Rate Clerks
<code>Bookkeeping_accounting_and_auditing_clerks</code>	Bookkeeping, Accounting, Auditing Clerks
<code>Cabinet_makers_and_bench_carpenters</code>	Cabinetmakers and Bench Carpenters
<code>Construction_laborers</code>	Construction laborers
<code>File_clerks</code>	File Clerks
<code>Gem_and_diamond_workers</code>	Gem and Diamond Workers
<code>Insurance_claims_clerks</code>	Insurance Claims Clerks
<code>Insurance_policy_processing_clerks</code>	Insurance Policy Processing Clerks
<code>Landscaping_and_Groundskeeping_Workers</code>	Landscaping and Groundskeeping Workers
<code>Postal_service_clerks</code>	Postal Service Clerks
<code>Receptionist_and_information_clerks</code>	Receptionists and Information Clerks
<code>Travel_guides</code>	Travel guides
<code>Word_Processors_and_Typists</code>	Word Processors and Typists

Tabella 4.2: Professioni O*NET presenti nell'ontologia

4.2.3 Estrazione dinamica

Le professioni non sono hardcodate nello script, ma vengono estratte dinamicamente dall'ontologia tramite la funzione `extract_ontology_jobs()`. Per ogni classe nel namespace `JobList` vengono raccolti tre predicati:

- `rdfs:label` – nome canonico della professione
- `01:Net_job_titles` – titoli alternativi riportati da O*NET (es. “Accounting Clerk, Accounting Specialist, ...”)
- `01:Net_short_description` – descrizione sintetica dell'attività lavorativa

Il prefisso “Sample of reported job titles:” viene rimosso automaticamente da `Net_job_titles` prima del matching, poiché altrimenti frammenti di testo descrittivo finirebbero nel confronto lessicale causando match spuri.

Capitolo 5

Strategie di matching

5.1 Strategia 1 – String Matching

5.1.1 Descrizione

La Strategia 1 è una baseline lessicale che non usa modelli di linguaggio. Confronta il **titolo del job nel DB** con il label canonico e tutti i titoli alternativi O*NET di ogni professione nell'ontologia, usando tre metriche combinate linearmente.

Nota

La Strategia 1 usa **solo il titolo** del job, non la descrizione. Questo la rende veloce ma insensibile al contenuto semantico delle attività lavorative.

5.1.2 Metriche

Similarità di Jaccard

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (5.1)$$

dove A e B sono gli insiemi di parole (dopo normalizzazione in minuscolo) dei due titoli. Misura la proporzione di parole condivise sul totale delle parole distinte. Insensibile all'ordine delle parole.

Overlap Coefficient

$$O(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)} \quad (5.2)$$

Divide per la cardinalità del set più piccolo invece che per l'unione. È più generoso di Jaccard quando un titolo breve è semanticamente contenuto in uno più lungo (es. “Data entry” contenuto in “Data Entry Operator”).

Similarità di Levenshtein normalizzata

$$L(s_1, s_2) = 1 - \frac{d_{edit}(s_1, s_2)}{\max(|s_1|, |s_2|)} \quad (5.3)$$

dove d_{edit} è la distanza di edit (numero minimo di inserimenti, cancellazioni e sostituzioni di caratteri). È l'unica delle tre metriche sensibile all'ordine dei caratteri – utile per abbreviazioni e varianti ortografiche.

Score combinato

$$\text{score}_{S1}(a, b) = 0.5 \cdot J(a, b) + 0.3 \cdot O(a, b) + 0.2 \cdot L(a, b) \quad (5.4)$$

I pesi assegnano priorità a Jaccard come metrica principale, con Overlap come supporto per asimmetrie di lunghezza e Levenshtein per variazioni ortografiche.

5.1.3 Implementazione

```

1 def s1_match_one(title: str, onto_jobs: list[dict]) -> dict | None:
2     best, bj, bv = 0.0, None, None
3     for j in onto_jobs:
4         cand = [j["label"]]
5         if j["titles"]:
6             cand += [t.strip() for t in j["titles"].split(",") if
7                     t.strip()]
8         for c in cand:
9             sc = _combined(title, c)
10            if sc > best:
11                best, bj, bv = sc, j, c
12 if best >= S1_THRESHOLD:
13     return {"job": bj, "score": best, "via": bv}
14 return None

```

Listing 5.1: Funzione di matching S1

La chiave "via" nel dizionario restituito indica esattamente quale label specifica (tra quelle alternative O*NET) ha prodotto lo score massimo – informazione utile per il debugging e per la valutazione della qualità del match.

5.2 Cosine Similarity – funzione condivisa

Tutte e tre le strategie NLP usano la stessa funzione per calcolare la similarità coseno, garantendo uniformità metodologica nei confronti.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} \quad (5.5)$$

dove $\mathbf{A}$ è il vettore embedding del job nel DB e $\mathbf{B}$ è il vettore embedding di una professione O*NET. Il risultato è compreso tra -1 e 1 ; per embedding di modelli linguistici il range effettivo è $[0, 1]$.

```

1 def cosine_similarity_matrix(query_emb: np.ndarray,
2                             corpus_embs: np.ndarray) -> np.ndarray:
3     query_norm = query_emb / (np.linalg.norm(query_emb) + 1e-9)
4     corpus_norm = corpus_embs / (
5         np.linalg.norm(corpus_embs, axis=1, keepdims=True) + 1e-9
6     )
7     return corpus_norm.dot(query_norm)

```

Listing 5.2: Implementazione vettorializzata della cosine similarity

Il termine $+10^{-9}$ (epsilon di stabilità numerica) previene la divisione per zero nel caso teorico in cui un vettore abbia norma nulla. La forma vettorializzata calcola in un'unica operazione la similarità tra il vettore query e tutti i 13 vettori O*NET, senza loop Python.

5.3 Strategia 2 – BERT base

5.3.1 Modello

Parametro	Valore
Identificatore HuggingFace	bert-base-uncased
Numero di parametri	110 milioni
Architettura	Transformer encoder, 12 layer
Dimensione embedding	768
Pre-training	Wikipedia EN + BookCorpus
Task di pre-training	Masked LM + Next Sentence Prediction

Tabella 5.1: Caratteristiche di bert-base-uncased

5.3.2 Strategia di embedding

BERT non produce nativamente un embedding di frase. Per ricavarlo si utilizza il vettore del token speciale [CLS], inserito automaticamente all'inizio di ogni sequenza di input. Questo token è stato progettato durante il pre-training per catturare una rappresentazione aggregata dell'intera sequenza.

```

1 def get_bert_embedding(text, tokenizer, model, max_length=512):
2     inputs = tokenizer(text, return_tensors="pt",
3                       truncation=True, max_length=max_length,
4                       padding=True)
5     with torch.no_grad():
6         outputs = model(**inputs)
7     # shape: [batch=1, seq_len, 768] -> [768]
8     cls_embedding = outputs.last_hidden_state[:, 0,
9     :].squeeze().numpy()
10    return cls_embedding

```

Listing 5.3: da BERT] Estrazione embedding [CLS] da BERT

`torch.no_grad()` disabilita il calcolo del gradiente poiché il modello è usato solo per inferenza, riducendo l'uso di memoria.

5.3.3 Limitazioni: anisotropy

BERT non è stato fine-tuned per la sentence similarity. I suoi embedding tendono a occupare una zona ristretta dello spazio vettoriale (fenomeno noto come *anisotropy*): i vettori puntano tutti in direzioni simili, rendendo la cosine similarity sistematicamente alta indipendentemente dalla reale similarità semantica. In pratica, con soglia 0.70, tutti i 9 job del dataset ottengono un match – il che non è necessariamente corretto.

5.4 Strategia 3 – BioBERT

5.4.1 Modello

Parametro	Valore
Identificatore HuggingFace	dmis-lab/biobert-base-cased-v1.2
Numero di parametri	110 milioni
Architettura	Transformer encoder, 12 layer (come BERT)
Dimensione embedding	768
Pre-training	BERT base + PubMed abstracts + PMC full text
Corpus aggiuntivo	4.5 miliardi di parole di letteratura biomedica

Tabella 5.2: Caratteristiche di BioBERT

5.4.2 Ragione dell’inclusione

BioBERT è incluso per **confronto metodologico**, non perché sia il modello migliore per questo task. La sua presenza consente di osservare empiricamente come la specializzazione di dominio influenzi i risultati: un modello addestrato su letteratura medica produce embedding diversi da BERT base quando processa testi di ambito lavorativo-amministrativo.

5.4.3 Risultati osservati

I test mostrano che BioBERT mappa 8 job su 9 sulla stessa classe (*File Clerks*), indipendentemente dal contenuto. Questo comportamento è attribuibile al disallineamento tra il dominio di addestramento (letteratura biomedica) e il dominio del task (job matching). La classe *File Clerks* ha una descrizione O*NET breve e generica che produce un vettore “neutro” vicino a molti testi in uno spazio semantico non calibrato per questo dominio.

Attenzione

L’analisi delle caratteristiche architettoniche di BioBERT, unite alle prestazioni deludenti rilevate in fase sperimentale, suggerisce l’inefficacia del suo impiego nel caso in esame. Contrariamente a quanto osservato con il modello BERT standard, la specificità del pre-addestramento di BioBERT su domini biomedici non ha prodotto i benefici attesi, rendendo superflua e non proficua qualsiasi ulteriore attività di fine-tuning.

5.5 Strategia 4 – MiniLM

5.5.1 Modello

Parametro	Valore
Identificatore HuggingFace	sentence-transformers/all-MiniLM-L6-v2
Numero di parametri	22 milioni
Architettura	Transformer encoder, 6 layer (distillato da BERT)
Dimensione embedding	384
Fine-tuning	Sentence similarity su 1+ miliardo di coppie
Libreria	sentence-transformers

Tabella 5.3: Caratteristiche di MiniLM

5.5.2 Differenze rispetto a BERT raw

MiniLM è stato addestrato con un obiettivo esplicito di sentence similarity tramite architettura *Siamese*: due reti con pesi condivisi processano coppie di frasi e vengono ottimizzate per produrre vettori vicini per frasi semanticamente simili e vettori lontani per frasi semanticamente diverse. Il risultato è che i suoi score di cosine similarity sono **calibrati**: uno score di 0.86 indica genuina similarità semantica, uno score di 0.30 indica genuina dissimilarità.

5.5.3 Implementazione

```

1 model = SentenceTransformer("all-MiniLM-L6-v2")
2 onto_embs = model.encode(onto_texts, convert_to_numpy=True)
3
4 for i, row in enumerate(db_jobs):
5     q_emb = model.encode(db_texts[i], convert_to_numpy=True)
6     # Cosine similarity calcolata con la stessa funzione di S2/S3
7     sims = cosine_similarity_matrix(q_emb, onto_embs)
8     idx = int(np.argmax(sims))
9     sc = float(sims[idx])
10    m = {"job": onto_jobs[idx], "score": sc} if sc >= threshold else
        None

```

Listing 5.4: Matching con MiniLM

Gli embedding O*NET vengono pre-calcolati una sola volta prima del loop sui job del DB, ottimizzando le prestazioni. Nonostante MiniLM fornisca internamente funzioni di cosine similarity, la funzione `cosine_similarity_matrix()` viene usata esplicitamente per uniformità con S2 e S3.

5.5.4 Input testuale

Sia per i job del DB che per le professioni O*NET, il testo passato al modello è una concatenazione di più campi:

- **Job nel DB:** `title + ". " + description`
- **Professione O*NET:** `label + Net_job_titles + Net_short_description`

Più contesto semantico viene fornito, più precisi risultano gli embedding prodotti dal modello.

Capitolo 6

Produzione dei risultati in formato Excel

6.1 Scopo dell'output Excel

A differenza della fase precedente, il modulo attuale non genera un grafo RDF A-Box, ma produce un report analitico in formato Microsoft Excel (.xlsx). Questa scelta è motivata dalla necessità di validare le soglie di confidenza e confrontare le diverse strategie di matching (lessicali e neurali) in modo leggibile e strutturato prima della definitiva persistenza nel Knowledge Graph.

Il file viene generato automaticamente nella cartella `output/` con il nome `rientra_matching.xlsx`.

6.2 Struttura del Workbook

Il documento Excel è organizzato in tre fogli di lavoro (sheet) distinti, ognuno con un ruolo specifico nella pipeline di valutazione dei risultati:

1. **Matching Results:** Foglio principale che contiene i risultati di tutte le strategie di matching per ogni job del database.
2. **Ontology Jobs:** Elenco delle professioni O*NET estratte dall'ontologia originale, utilizzato come riferimento.
3. **Score Detail:** Matrice dettagliata che riporta gli score raw per ogni combinazione job-modello.

6.3 Dettaglio del foglio “Matching Results”

Il foglio principale raccoglie le informazioni provenienti dal database PostgreSQL e le arricchisce con i risultati dei modelli NLP.

6.3.1 Metadati e colonne

Per ogni riga (corrispondente a un job del DB), vengono riportati:

- **Dati DB:** ID (`ext_job`), Titolo e Descrizione.

- **IRI Individuo:** L'URI che identificherebbe l'entità nel grafo (`rientra:ExtJob_*`).
- **S1 String Match:** Label trovata, punteggio e etichetta specifica utilizzata per il match.
- **Strategie NLP (S2, S3, S4):** Professione identificata, score di *cosine similarity* e URI della classe corrispondente.

6.3.2 Logica del Verdetto

L'ultima colonna del foglio fornisce un **Verdetto** sintetico basato sul consenso tra i modelli NLP.

Verdetto	Descrizione
Tutti concordano	Tutti i modelli NLP attivi hanno identificato lo stesso URI O*NET.
Accordo parziale	Esiste un consenso tra alcuni modelli o lo score è considerato affidabile.
Divergono	I modelli NLP puntano a professioni O*NET differenti.
Solo S1	Solo lo string matching ha prodotto un risultato sopra soglia.
Nessun match	Nessuna strategia ha superato le soglie di confidenza configurate.

Tabella 6.1: Logica del verdetto nel foglio Excel

6.4 Stili e Formattazione Condizionale

Per facilitare l'analisi visiva, il modulo applica stili specifici basati sulla qualità del punteggio:

- **Colorazione per Strategia:** Ogni colonna di matching ha un colore distintivo (es. giallo per S1, verde per MiniLM).
- **Evidenziazione Score:** I punteggi superiori alle soglie (≥ 0.70 per BERT e ≥ 0.50 per MiniLM) sono evidenziati in grassetto verde.
- **Sotto Soglia:** I risultati insufficienti riportano il valore *raw* in rosso con la dicitura “sotto soglia”.

6.5 Esempio di implementazione

Di seguito è riportata la struttura della funzione Python che utilizza la libreria `openpyxl`:

```
1 def save_excel(db_jobs, s1_res, nlp_results, model_names, onto_jobs):
2     wb = Workbook()
3     ws1 = wb.active
4     ws1.title = "Matching Results"
5     # Configurazione header e stili
6     for r_idx, row in enumerate(db_jobs, start=2):
7         # Scrittura dati DB e risultati strategie
8         # Calcolo del verdetto di consenso
9     wb.save(os.path.join(OUTPUT_DIR, "rientra_matching.xlsx"))
```

Listing 6.1: Snippet della funzione `save_excel`

Capitolo 7

Output e interpretazione dei risultati

7.1 Tabella comparativa

La sezione 6 dell'output mostra una tabella con una riga per ogni job del DB e una coppia di colonne (match + score) per ogni strategia disponibile. La colonna **Verdetto** riassume l'accordo tra i modelli NLP:

Verdetto	Significato	Affidabilità
✓ tutti	Tutti i modelli NLP concordano	Alta
~ parz.	Accordo parziale (alcuni modelli)	Media
≠ div.	I modelli puntano a professioni diverse	Bassa
S1 only	Solo string matching ha un match	Verificare manualmente
× no	Nessun match sopra soglia	Job non presente in ontologia

Tabella 7.1: Interpretazione della colonna Verdetto

Nota

Il verdetto è calcolato confrontando gli URI delle professioni trovate, non gli score. Due modelli con score molto diversi (es. 0.929 e 0.863) che puntano alla stessa professione danno verdetto “✓ tutti”. Due modelli con score simili che puntano a professioni diverse danno “≠ div.”.

7.2 Risultati correnti

Con le 13 professioni attualmente nell'ontologia e le soglie configurate:

Job	S4 MiniLM	Match	Affidabilità
ext01 Accounting clerk	0.863	Bookkeeping Clerks	Alta
ext05 Data entry employee	0.556	Bookkeeping Clerks	Discutibile
ext09 Supermarket cashier	0.511	Billing Clerks	Discutibile
ext02-04, 06-08	< 0.50	Nessun match	Professione assente

Tabella 7.2: Risultati MiniLM con soglia 0.50

Il solo match semanticamente affidabile è **ext01**. Gli altri 6 job non trovano corrispondenza perché le loro professioni (Data analyst, Data scientist, Content manager web, ecc.) non sono presenti tra le 13 classi dell'ontologia.

Capitolo 8

Limitazioni note e sviluppi futuri

8.1 Limitazioni correnti

8.1.1 Vocabolario ontologico insufficiente

Il collo di bottiglia principale non è la qualità dei modelli NLP ma la limitata copertura del vocabolario O*NET nell'ontologia: 13 professioni su migliaia disponibili. L'aggiunta delle professioni mancanti (Data analyst, Data scientist, Content manager, ecc.) migliorerà drasticamente i risultati di MiniLM.

8.1.2 Soglie BERT non calibrate

Con soglia 0.70, BERT e BioBERT trovano un match per tutti i 9 job indipendentemente dalla qualità. La soglia va alzata a ≥ 0.90 come raccomandato dal referente del progetto.

8.1.3 BioBERT – dominio errato

BioBERT è stato incluso per confronto metodologico ma non è adatto al task di job matching. Produce risultati non affidabili (8/9 job mappati su *File Clerks*) e potrebbe essere sostituito con un modello più appropriato per il dominio lavorativo.

8.2 Sviluppi pianificati

1. **Aggiornamento soglie:** portare BERT_THRESHOLD a 0.92 e aggiungere la colonna “gap” (differenza tra primo e secondo score) come misura di robustezza del match.
2. **Arricchimento ontologia:** aggiungere le 10 nuove professioni O*NET che coprono i job attualmente senza match.
3. **Modello modificato:** Si è scelto di superare la mappatura statica tra i framework O*NET ed ESCO, optando per l'implementazione di un modello BERT addestrato su dataset reali. Tale approccio è finalizzato all'individuazione accurata di equivalenze e similitudini semantiche tra i profili.

Capitolo 9

Guida all'uso

9.1 Prerequisiti

1. PostgreSQL 17 attivo e accessibile su `localhost:5432`
2. Database `rientra_db` creato: `createdb -U postgres rientra_db`
3. Tabella `ext_job` popolata: `psql -U postgres -d rientra_db -f rientra_ext_jobs.sql`
4. File `Rientra.rdf` nella stessa directory dello script
5. Ambiente virtuale Python attivato

9.2 Installazione

```
1 python3 -m venv venv
2 source venv/bin/activate           # macOS/Linux
3 # oppure: venv\Scripts\activate    # Windows
4
5 pip install psycpg2-binary rdflib scikit-learn rich numpy
6 pip install transformers torch sentence-transformers
```

Listing 9.1: Setup ambiente

9.3 Esecuzione

```
1 python3 rdb2rdf_step2.py # macOS
2
3 python rdb2rdf_step2.py # Windows
```

Listing 9.2: Avvio dello script

9.4 Output atteso

Al termine dell'esecuzione viene prodotto nella cartella `output/`:

- `rientra_matching.xlsx` – file excel

L'esecuzione completa con tutti i modelli NLP richiede circa 2–5 minuti alla prima esecuzione (download dei modelli da HuggingFace) e circa 30–60 secondi nelle esecuzioni successive (cache locale).

Appendice A

Messaggi di warning noti

Durante l'esecuzione compaiono alcuni messaggi di warning che sono normali e non indicano errori:

Messaggio	Causa e comportamento
<code>cls.predictions.* UNEXPECTED</code>	Layer di pre-training (Masked LM) non usati. Vengono scartati senza effetti sul funzionamento.
<code>cls.seq_relationship.* UNEXPECTED</code>	Layer Next Sentence Prediction non usato. Scartato.
<code>embeddings.position_ids UNEXPECTED</code>	Buffer deprecato nelle versioni recenti di Transformers. Ignorato.
<code>Unauthenticated requests to HF Hub</code>	Modelli scaricati senza token HuggingFace. Funziona, ma con rate limiting più restrittivo. Risolvibile impostando <code>HF_TOKEN</code> .

Tabella A.1: Warning noti e loro interpretazione

Appendice B

Glossario

A-Box (Assertional Box)

La parte del grafo RDF contenente gli individui (istanze), in contrapposizione alla T-Box che contiene le definizioni di classi e proprietà.

Anisotropy

Fenomeno per cui gli embedding prodotti da BERT raw tendono a occupare una regione ristretta dello spazio vettoriale, con tutte le direzioni simili tra loro. Causa score di cosine similarity sistematicamente alti per qualsiasi coppia di testi.

CLS token

Token speciale [CLS] inserito da BERT all'inizio di ogni sequenza di input. Il suo vettore nell'ultimo hidden state viene usato come embedding dell'intera sequenza.

Cosine Similarity

Misura di similarità tra due vettori basata sull'angolo che formano, indipendentemente dalla loro magnitudine.

Embedding

Rappresentazione vettoriale densa di un testo in uno spazio numerico ad alta dimensionalità, prodotta da un modello neuronale. Testi semanticamente simili hanno embedding vicini.

Entity Linking

Task di NLP che consiste nel collegare menzioni testuali di entità a voci corrispondenti in una base di conoscenza strutturata.

Fine-tuning

Processo di ri-addestramento di un modello pre-addestrato su un dataset specifico per un task specifico, aggiornando parzialmente i pesi del modello.

Knowledge Graph

Grafo strutturato che rappresenta entità e le relazioni tra esse in forma di triple soggetto-predicato-oggetto.

O*NET

Occupational Information Network: database del Dipartimento del Lavoro degli USA che fornisce descrizioni standardizzate di migliaia di professioni, incluse competenze, attività e titoli alternativi.

RDB2RDF

Standard W3C per la mappatura di dati relazionali (database SQL) in dati RDF. Include il Direct Mapping e R2RML.

T-Box (Terminological Box)

La parte del grafo RDF (o dell'ontologia OWL) contenente le definizioni di classi, proprietà e assiomi – in contrasto con l'A-Box che contiene gli individui.

Triple RDF

Unità atomica del modello RDF, nella forma *soggetto – predicato – oggetto*.